CS/EE 120B Custom Laboratory Project Report

Morse Code Game

Angelo Torres

6/12/25

Introduction

In the Morse Code Challenge, the microcontroller sends a random letter in Morse code using an LED blink and a buzzer tone. The player decodes the letter by cycling through the options with two buttons. After confirming their choice, the system informs the player whether their answer is correct, updates their score, and moves on to the next round. After 5 rounds, a win/lose message will be displayed: “You Win!” if the player answers 3 or more correctly, or “Game Over” if they do not.

Build-Upons

Character LCD
16×2 LCD to display letters and user messages. Additionally, define at least two custom characters for a dot (“·”) and a dash (“–”)

Passive Buzzer with Dynamic Tones
Generate precise 600 Hz tones in software for both dot and dash signals, along with distinct jingles to provide feedback for “correct” and “incorrect” responses.

EEPROM High-Score Storage
After each game, compare the current score to the stored high score in the on-chip EEPROM and update it if the new score is higher.

74HC595 IC
shift register

User Guide

Letter Selection and Transmission:
- A random letter (A–Z) is selected and transmitted in Morse code.
- The LED blinks with short pulses (dot) or long pulses (dash).
- The buzzer produces simultaneous sounds.

Player Input:
- Players decode the Morse code and input their guess.
- Press the “Next Letter” button to advance a numerical code (1–26) displayed on the 7-segment display.
- Press “Confirm” to lock in the guess.

Evaluation and Scoring:
- The player’s guess is compared to the random letter.
- The score counter on the 7-segment display is incremented based on correct guesses.

Round Management:
- The game consists of 5 rounds.
- After round 5, the display will show “Win” or “Lose” outputted on the LCD

Hardware Components Used

- ATmega328P
- 2 Pushbuttons:
  - PD2: “Next Letter”
  - PD3: “Confirm Selection”
- 4-Digit 7-Segment Display:
  - Segment Bits: PB0–PB3
  - Digit Selects: PC0–PC3
- Single Indicator LED:
  - PB5 (for Morse code flashes)
- Passive Buzzer:
  - PC4 (for tone output)
- 16×2 LCD1602 Module:
  - 6-pin interface (on PC5, PC4, PD4,PD5, PD6, PD7)
- 74HC595 IC

Software Libraries Used

<avr/io.h>: includes the definitions for all relevant hardware registers and pin names associated with the used AVR microcontroller. This file is crucial for the project as it allows the code to interact directly with the hardware to manage the LCD, read button inputs, and control the buzzer and LED.

<avr/interrupt.h>: facilitates the management of hardware interrupts. It includes the ISR() macro to define the interrupt service routine for the timer and the sei() function for enabling interrupts, which are vital for handling the timing and multiplexing of the display in the project.

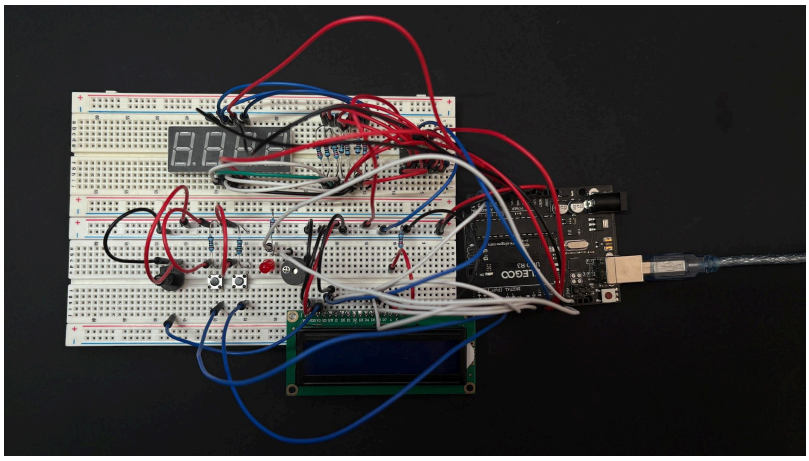
<util/delay.h>: offers functions that create precise software delays, especially _delay_ms(). This is utilized throughout the code to manage the timing of Morse code signals, the pauses between characters, and how long messages are displayed on the screen.

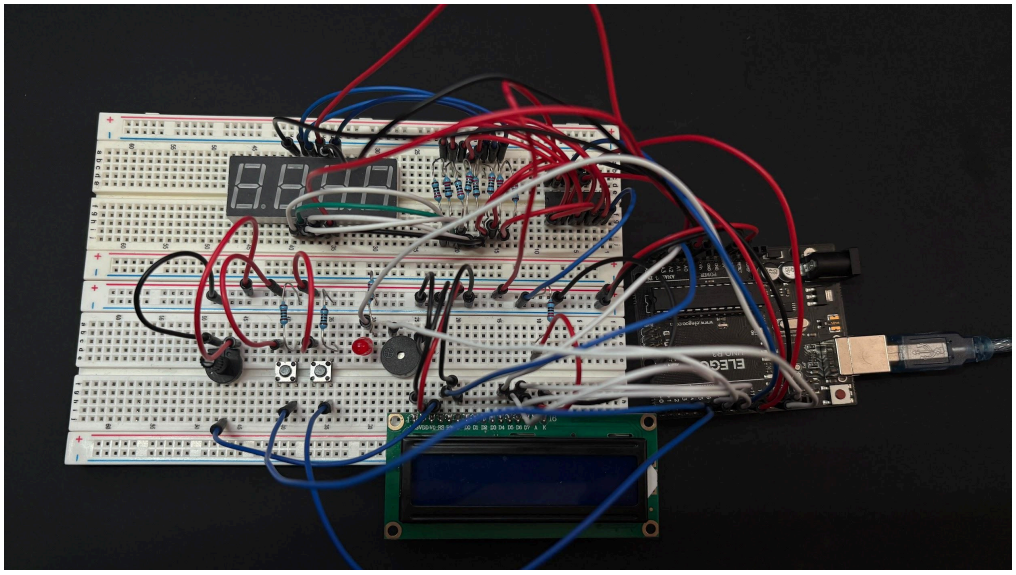
<avr/eeprom.h>: provides functions for reading from and writing to the microcontroller's EEPROM memory. This capability allows for the storage of the high score and random number seed, ensuring their retention when the device is powered off.

<stdlib.h>: used for generating pseudo-random numbers using the srand() and rand() functions, producing a random sequence of Morse code characters for the game.

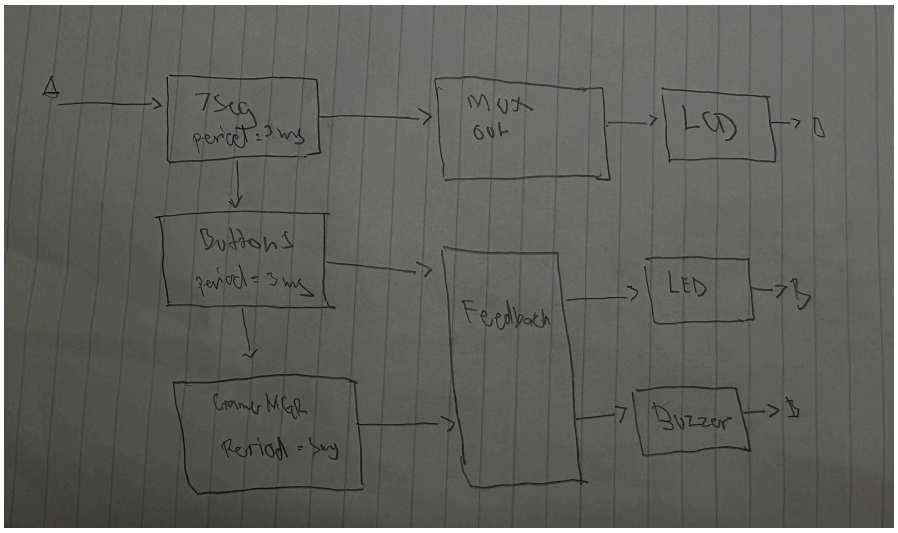
<stdio.h>: provides the sprintf() function, which formats strings by combining static text with variables such as the score and round number for display on the LCD.

Wiring Diagram

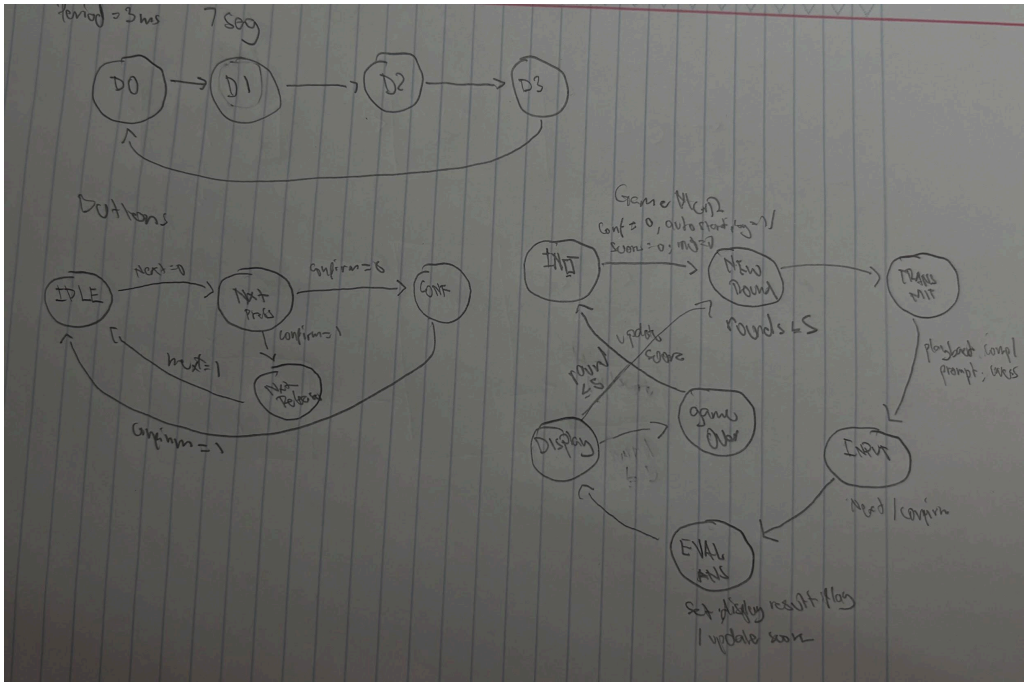




Task Diagram



SynchSM Diagram


period = 3 ms
7 seg
D0
D1
D2
D3
Buttons
IDLE
next=0
Next prob
confirm=5
CONF
confirm=1
next=1
Next Problem
confirm=1
Game Mgmt
conf = 0, auto start (y=1)
score = 0; mul=0
INIT
NEW Round
rounds ≤ 5
TRANS MIT
playback compl
prompt, LEDs
update score
round ≤ 5
Display
game Over
INPUT
next / confirm
EVAL ANS
set display result flag
/ update score